

RISC-V Cheatsheet

Arithmetic			
Name	Syntax	Result	IF
add	add rd, rs1, rs2	$rd = rs1 + rs2$	R
addi	addi rd, rs1, imm	$rd = rs1 + imm$	I
sub	sub rd, rs1, rs2	$rd = rs1 - rs2$	R
lui	lui rd, imm	$rd = imm \ll 12$	I
neg	neg rd, rs1	$rd = -rs1$	(p)
li	li rd, imm	$rd = imm$	(p)
mv	mv rd, rs1	$rd = rs1$	(p)
nop	nop	—	(p)

Logical			
Name	Syntax	Result	IF
and	and rd, rs1, rs2	$rd = rs1 \ \& \ rs2$	R
andi	andi rd, rs1, imm	$rd = rs1 \ \& \ imm$	I
or	or rd, rs1, rs2	$rd = rs1 \ \ rs2$	R
ori	ori rd, rs1, imm	$rd = rs1 \ \ imm$	I
xor	xor rd, rs1, rs2	$rd = rs1 \oplus rs2$	R
xori	xori rd, rs1, imm	$rd = rs1 \oplus imm$	I
not	not rd, rs1	$rd = \sim rs1$	(p)

Shifts			
Name	Syntax	Result	IF
sll	sll rd, rs1, rs2	rd = rs1 $\ll$ rs2	R
slli	slli rd, rs1, imm	rd = rs1 $\ll$ imm	I
srl	srl rd, rs1, rs2	rd = rs1 $\gg$ rs2	R
srli	srli rd, rs1, imm	rd = rs1 $\gg$ imm	R
sra	sra rd, rs1, rs2	rd = rs1 $\ggg$ rs2	I
srai	srai rd, rs1, imm	rd = rs1 $\ggg$ imm	I

Compares			
Name	Syntax	Result	IF
slt	slt rd, rs1, rs2	rd = (rs1 < rs2)	R
slti	slti rd, rs1, imm	rd = (rs1 < imm)	I
sltu	sltu rd, rs1, rs2	rd = (rs1 < rs2)	R
sltiu	sltiu rd, rs1, imm	rd = (rs1 < imm)	I
seqz	seqz rd, rs1	rd = (rs1 == 0)	(p)
snez	snez rd, rs1	rd = (rs1 $\neq$ 0)	(p)
sltz	sltz rd, rs1	rd = (rs < 0)	(p)
sgtz	sgtz rd, rs1	rd = (rs1 > 0)	(p)

Multiplication			
Name	Syntax	Result	IF
mul	mul rd, rs1, rs2	rd = (rs1 * rs2)[31:0]	R
mulh	mulh rd, rs1, rs2	rd = (rs1 * rs2)[63:32]	R
mulhu	mulhu rd, rs1, rs2	rd = (rs1 * rs2)[63:32]	R
mulhsu	mulhsu rd, rs1, rs2	rd = (rs1 * rs2)[63:32]	R

Division			
Name	Syntax	Result	IF
div	div rd, rs1, rs2	rd = rs1 / rs2	R
divu	divu rd, rs1, rs2	rd = rs1 / rs2	(p)
rem	rem rd, rs1, rs2	rd = rs1 % rs2	R
remu	remu rd, rs1, rs2	rd = rs1 % rs2	(p)

(p)	pseudo-instruction	rs1	source register 1
imm	immediate value	rs2	source register 2
pc	program counter	rd	destination register

Branches			
Name	Syntax	Result	IF
beq	beq rs1, rs2, imm	if(rs1 == rs2): pc += imm	SB
bne	bne rs1, rs2, imm	if(rs1 $\neq$ rs2): pc += imm	SB
blt	blt rs1, rs2, imm	if(rs1 < rs2): pc += imm	SB
bltu	bltu rs1, rs2, imm	if(rs1 < rs2): pc += imm	SB
bge	bge rs1, rs2, imm	if(rs1 $\geq$ rs2): pc += imm	SB
bgeu	bgeu rs1, rs2, imm	if(rs1 $\geq$ rs2): pc += imm	SB
auipc	auipc rd, imm	rd = pc + (imm $\ll$ 12)	U
beqz	beqz rs1, imm	if(rs1 == 0): pc += imm	(p)
bnez	bnez rs1, imm	if(rs1 $\neq$ 0): pc += imm	(p)
bltz	bltz rs1, imm	if(rs1 < 0): pc += imm	(p)
bgt	bgt rs1, rs2, imm	if(rs1 > rs2): pc += imm	(p)
bgtu	bgtu rs1, rs2, imm	if(rs1 > rs2): pc += imm	(p)
bgtz	bgtz rs1, imm	if(rs1 > 0): pc += imm	(p)
ble	ble rs1, rs2, imm	if(rs1 $\leq$ rs2): pc += imm	(p)
bleu	bleu rs1, rs2, imm	if(rs1 $\leq$ rs2): pc += imm	(p)
blez	blez rs1, imm	if(rs1 $\leq$ 0): pc += imm	(p)
bgez	bgez rs1, imm	if(rs1 $\geq$ 0): pc += imm	(p)

Jumps			
Name	Syntax	Result	IF
j	j imm	pc += imm	(p)
jal	jal rd, imm	rd = pc+4; pc += imm	UJ
jalr	jalr rd, rs1, imm	rd = pc+4; pc = rs1+imm	UJ

Functions			
Name	Syntax	Result	IF
call	call label	ra = pc + 4; pc = &label	(p)
ret	ret	pc = ra	(p)

Loads			
Name	Syntax	Result	IF
lw	lw rd, imm(rs1)	rd = mem[rs1+imm]	I
lh	lh rd, imm(rs1)	rd = mem[rs1+imm][0:15]	I
lhu	lhu rd, imm(rs1)	rd = mem[rs1+imm][0:15]	I
lb	lb rd, imm(rs1)	rd = mem[rs1+imm][0:7]	I
lbu	lbu rd, imm(rs1)	rd = mem[rs1+imm][0:7]	I
la	la rd, symbol	rd = &symbol	I

Stores			
Name	Syntax	Result	IF
sw	sw rs2, imm(rs1)	mem[rs1+imm] = rs2	S
sh	sh rs2, imm(rs1)	mem[rs1+imm][0:15] = rs2	S
sb	sb rs2, imm(rs1)	mem[rs1+imm][0:7] = rs2	S

Registers		
Register	Alias	Description
x0	zero	hard-wired zero
x1	ra	return address
x2	sp	stack pointer
x3	gp	global pointer
x4	tp	thread pointer
x5-7	t0-2	temporaries
x8	s0/fp	saved register / frame pointer
x9	s1	saved register
x10-11	a0-1	function arguments / return values
x12-16	a2-11	function arguments
x18-27	s2-11	saved registers
x28-31	t3-6	temporaries

Type	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	func7								rs2				rs1				func3				rd				opcode							
I	imm[11:0]												rs1				func3				rd				opcode							
S	imm[11:5]								rs2				rs1				func3				imm[4:0]				opcode							
SB	imm[12]		imm[10:5]						rs2				rs1				func3				imm[4:1]		imm[11]		opcode							
U	imm[32:12]																				rd				opcode							
UJ	imm[20]		imm[10:1]								imm[11]		imm[19:12]								rd				opcode							